

CMU AGENTIC SYSTEMS STUDIO · TRACK A — TECHNICAL BUILD

Phase 2: Architecture, Prototype & Evaluation Plan

The AI Bidding War: Autonomous Procurement & Vendor Negotiation

Asli Gulcur (solo project)

April 2026

CONTENTS

- Executive Summary
- 1. Problem Statement
- 2. Why This Is An Agentic Problem
- 3. Architecture Concept, Coordination Logic & State
 - Control Plane & Runtime
 - Strategic Advisor
 - State Management Across Negotiation Rounds
 - Coordination Logic — Who Starts, Handoffs, Stopping
 - CFO Portfolio Narrative (Offline)
- 4. Why This Architecture Beats A Simpler Alternative
- 5. Scope, Boundaries, And Governance
 - Scope
 - Risk Register And Mitigations
- 6. Tools, Memory, And Data Design
 - Per-Role Information Access
- 7. Evaluation Plan
 - Two Evaluation Layers
 - Constants
 - Per-Scenario Baselines (All Eleven)
- 8. Prototype & Traces
- Appendix A: Multi-Agent Canvas — System View
- Appendix B: Agent Canvases (Detail)
- Appendix C: Deliverables And Submission Checklist
- References

Phase 2: Architecture, Prototype & Evaluation Plan

The AI Bidding War: Autonomous Procurement & Vendor Negotiation

Asli Gulcur (solo project) — April 2026

Track: Track A — Technical Build (CMU Agentic Systems Studio)

Executive Summary

Enterprise sourcing is slow and iterative. Industry benchmarks from WorldCC and APQC document multi-week bid-to-contract cycles, with negotiation occurring in more than 60% of transactions even when standard templates are used. Static bid collection does not resolve iterative trade-offs across price, timing, service levels, and compliance constraints. A single shared LLM chat cannot preserve vendor-side information asymmetry or separate negotiated intent from authorized spend.

This submission presents a hub-and-spoke multi-agent procurement simulation targeting Chief Procurement Officers, supply chain executives, and enterprise procurement teams. A Buyer negotiates with three isolated vendors (A, B, C) over up to three rounds, with Strategic Audit, human approval, and a deterministic spend guardrail enforcing governance at every award path. Isolation is enforced in code through separate data structures, not prompts alone, so adversarial game-theoretic structure is real rather than theatrical.

Executable awards must pass Strategic Audit, then human approval, then a deterministic ceiling check before any contract is written.

On the 2026-04-11 harvested snapshot, eleven scenarios were scored across two layers — governance and negotiation. No executed award exceeded \$15,000. Governance passed for all 11 scenarios. Negotiation scored nine pass, one fail-by-design (Scenario 2, the red-team guardrail stress test), and one partial (Scenario 6). Six scenarios resulted in executed contracts; five ended in documented `NO_AWARD` outcomes. Measured results, harvest tables, reproducibility notes, and failure/boundary case write-ups live in the supplementary report `evaluation/FINAL_EVALUATION_REPORT.md`.

Development approach. Phase 1 established scope, hub-and-spoke orchestration, the guardrail concept, and six archetypes. Phase 2 operationalizes state and handoffs and expands to eleven scenarios (IDs 1–11), including HIPAA/GDPR, rip-and-replace, bid steering, conflict-of-interest, and other boundary cases with explicit governance vs negotiation baselines, responding to instructor Phase 1 feedback.

1. Problem Statement

Enterprise RFP and contract negotiation is labor-intensive. Industry benchmarks quantify the gap between "collecting static bids" and "getting to signature," supporting the ROI case for dynamic, structured negotiation.

World Commerce & Contracting (WorldCC) Benchmark Report 2023 reports bid-to-contract elapsed times on the order of multiple weeks: approximately 4.4 weeks for low-complexity domestic cases, approximately 11.6 weeks for medium-complexity domestic cases, and approximately 24–29 weeks for high-complexity

domestic/international cases. The same study finds negotiation in more than 60% of transactions even when standard templates are used, so cycle time is not solved by bid portals alone.¹

APQC Open Standards Benchmarking publishes cross-industry median cycle times for procurement: 60 calendar days from internal need identification to signed contract for a sourcing event ($n \approx 3,080$ organizations), and 40 days from opening negotiation to signature when establishing a supplier contract ($n \approx 3,081$) — again consistent with multi-week coordination, not a single batch upload.^{2 3}

Separately from cycle-time statistics, bid or quote collection alone does not resolve iterative trade-offs across multiple attributes (e.g. price, timing or service levels, and policy or compliance constraints) — the kind of work WorldCC associates with frequent negotiation even when templates exist. The prototype here does not claim live production "real-time" integration; it is a round-based simulation that models structured multi-agent negotiation under those tensions, with vendor isolation, audit-before-approval on awards, and hard budget enforcement outside the LLM — addressing both efficiency and governance concerns that a single shared-context chat model does not solve.

Therefore the prototype simulates isolated multi-party negotiation, logs decisions and audits for review, and evaluates runs against stated criteria — Architecture through Evaluation Plan below spell out how.

2. Why This Is An Agentic Problem

Procurement negotiation is fundamentally a problem of **multiple principals with private information**. Each vendor has a floor price, a competitive strategy, and constraints they do not share with competitors. The buyer has budget targets and compliance requirements they do not fully reveal to vendors. This information asymmetry is not a modeling convenience — it is the structural feature that makes negotiation adversarial and game-theoretic.

A single shared LLM thread cannot genuinely negotiate with itself. Without structural separation, role incentives tend to collapse into one compromise — a phenomenon this project terms **context contamination**. When all vendor personas share the same context window, the model has access to every party's private information simultaneously. The result is not negotiation but arbitration: the model produces a "fair" middle-ground outcome rather than the adversarial dynamic that drives real procurement value. Role-playing instructions alone cannot prevent the model from implicitly using Vendor B's floor price when generating Vendor A's counteroffer.

This project addresses the problem through three structural separations. First, **vendor isolation**: each vendor operates in its own conversation thread, seeing only the buyer's public messages and never another vendor's bids or strategy. This is enforced by data structures in code (`vendor_histories`), not by prompt instructions alone. Second, **hub consolidation**: the buyer sees parsed bid summaries from all vendors each round but never raw vendor thread text, preserving the buyer's analytical advantage without leaking vendor-side reasoning. Third, **governance separation**: Strategic Audit and a human gatekeeper operate after negotiation, adding an assurance layer before any executable spend path. The deterministic `award_contract` guardrail sits outside the LLM entirely, ensuring that policy enforcement is not delegated to probabilistic model output.

3. Architecture Concept, Coordination Logic & State

The sections below specify runtime control, Advisor I/O, in-memory state, handoffs, and stopping rules — the implementation backbone for the agentic problem, scope, and governance commitments described above.

Control Plane & Runtime

- **CLI / operator:** `python orchestrator.py --id <N>` with $N \in 1-11$ per `config/scenarios.json`.
Optional custom RFP text or scenario requirements.
- **Hub-and-spoke orchestrator** (`orchestrator.py`): Schedules rounds; routes messages so each vendor sees only the buyer's public thread slice (RFP on round 1; buyer's latest hub message on later rounds) plus an isolation footer — not other vendors' bids or personas.
- **Anthropic API:** Buyer, vendors, and Strategic Advisor use configured Claude models via `.env` (`ANTHROPIC_API_KEY` , optional `ANTHROPIC_MODEL`).
- **Personas:** `agents/*/agent.md` (buyer standard/malicious, vendors A/B/C).
- **Deterministic guardrail:** `skills/award_contract.py` — hard-coded ceiling check (default \$15,000), acting as an immutable financial circuit breaker outside the LLM's probabilistic output: over-ceiling → ERROR, not execution.
- **Optional packaging:** `agents/*/agent.md` and `skills/` follow a common agent/skill layout for external tooling. Docker (`Dockerfile` , `docker-compose.yml`) is optional; default is venv + local Python (README).

Strategic Advisor (First-Class Governance Component)

The Strategic Advisor is not a bidding participant. It runs after the buyer proposes an award (parseable `AWARD_JSON` line) and before human approval / `award_contract` .

Field	Detail
Implementation	<code>agents/advisor.py</code> — <code>run_strategic_audit()</code> ; invoked from <code>orchestrator._emit_strategic_audit()</code> .
Inputs	Scenario metadata — dict from <code>config/scenarios.json</code> (name, requirements, description, budget, ...). RFP text — session RFP string. Buyer recommendation — buyer message containing <code>AWARD_JSON</code> and rationale. Serialized threads — <code>format_negotiation_history(vendor_histories, conversation_buyer)</code> (full buyer hub thread + isolated per-vendor threads).
Outputs	Markdown Value Audit: structured sections (e.g. Compliance Gaps, Value Leakage, Risk Assessment, Conflict of Interest & Process Integrity, Strategic Audit Summary) per <code>ADVISOR_SYSTEM_PROMPT</code> .
Failure mode	On API/runtime error, orchestrator substitutes <code>Advisor error: ...</code> stub text, still logs audit slot to scenario log and evidence JSON, and continues to human approval — traceable fallback, not silent failure.
Duplicate proposal (finalize)	In the finalize segment after rounds, if the buyer's <code>AWARD_JSON</code> matches the same vendor and price as the last Strategic Audit, the orchestrator may skip a redundant audit (see <code>orchestrator.py</code>).

Architecture diagrams: `docs/System Architecture Diagram/ai_bidding_war_system_architecture-6-2.svg` is the primary system canvas (companion HTML: `docs/Agent and Multi-Agent System Canvases/MA_ArchCanvas_The_AI_Bidding_War.html`) — Strategic Advisor appears in Zone 3 on that figure.

State Management Across Negotiation Rounds (Hub Consolidated View)

Isolation is enforced by data structures, not prompts alone:

State	Role
vendor_histories	<code>dict[vendor_id → list of {role, content} messages]</code> . Each vendor thread only receives: round 1 — RFP + scenario framing + isolation footer; rounds 2–3 — only <code>buyer_reply_prev</code> (buyer's previous hub assistant message), not other vendors' replies. Threads append each round.
conversation_buyer	Single cumulative buyer conversation. Each round the orchestrator appends a user message that includes: scenario id/name, full RFP, round index, <code>bid_summary</code> (JSON of parsed <code>BID_JSON</code> for all vendors this round), internal/target budget, hard ceiling text, and award instructions. Then the buyer model reply is appended as assistant. This is the buyer's consolidated view of competition — derived from parsed bids, not raw vendor prose from other threads.
buyer_reply_prev	After each buyer turn, stores the latest buyer assistant text for next round's vendor user prompts — only public buyer language is relayed to vendors.
Strategic Audit input	At award time, <code>format_negotiation_history</code> stitches full buyer thread plus all three vendor threads for governance context — this is not shown to vendors during negotiation.
Persistence	<code>logs/evidence_log_*.json</code> — structured events (round, actor, action, ...). <code>logs/scenario_*.log</code> — human-readable transcript including Strategic Audit blocks.

Coordination Logic — Who Starts, Handoffs, Stopping

Typical happy path: each round → vendors A→B→C submit bids → buyer sees parsed `bid_summary` → buyer may `CONTINUE` or propose `AWARD_JSON` → on award → Strategic Audit → human yes → `award_contract` . After `no` (while vendor rounds remain), optional nudge and re-audit may repeat until finalize or `NO_AWARD` . The table below is the authoritative step-by-step reference.

Stage	Behavior
Start	Operator runs <code>orchestrator.py --id N</code> ; orchestrator loads scenario + RFP; initializes <code>vendor_histories</code> , <code>conversation_buyer</code> , <code>buyer_reply_prev</code> .
Per round	Vendors A→B→C (isolated calls) → orchestrator parses <code>BID_JSON</code> → builds <code>bid_summary</code> → buyer user message → buyer assistant reply.
What happens next	If buyer outputs <code>CONTINUE</code> , next round (if under round cap). If <code>AWARD_JSON</code> → Strategic Audit → human yes/no → on yes, <code>award_contract</code> ; on no while vendor rounds remain, optional governance nudge (orchestrator relays short instruction to buyer; re-audit may run).
Finalize (after rounds)	If no contract yet, <code>orchestrator.py</code> runs a finalization segment: buyer must emit <code>AWARD_JSON</code> (within guardrail) or explicit <code>NO_AWARD</code> . Same audit → human → <code>award_contract</code> ordering applies; if human no, no governance nudge (vendor rounds exhausted — a nudge cannot change vendor terms).
Human-in-the-loop	After audit: terminal yes/no; <code>award_contract</code> only after yes.
Stopping	Max 3 negotiation rounds; finalize path after rounds; session may end with <code>NO_AWARD</code> , human veto, guardrail <code>ERROR</code> , or successful contract write to <code>logs/contract.txt</code> when applicable. No infinite loop: round counter and finalize path cap length.

CFO Portfolio Narrative (Offline)

Offline batch (not in the live loop): `scripts/cfo_monthly_report.py` aggregates `logs/evidence_log*.json` by month (or `--all`), calls the API with `agents/cfo_narrative/agent.md` , and writes `CF0 Reports/cfo/CF0_narrative_<YYYY-MM>.md` . Portfolio-level themes for CPO / finance / audit — same JSON as rubric scoring, different audience; not a substitute for the supplementary evaluation report. Example: `python scripts/cfo_monthly_report.py --month 2026-04` .

4. Why This Architecture Beats A Simpler Alternative

The most obvious simpler approach would be a single LLM thread with all vendors sharing one context window. The table below compares that design to the hub-and-spoke architecture implemented here.

Dimension	Single-thread approach	This design (hub-and-spoke)
Information asymmetry	All vendors share one context; the model has access to every party's private information. Role-playing instructions cannot prevent cross-contamination.	Vendor threads are structurally isolated via <code>vendor_histories</code> . Each vendor sees only the buyer's public messages. Enforced in code, not prompts.
Negotiation dynamics	Incentives collapse into a single compromise. The model arbitrates rather than negotiates — producing "fair" outcomes rather than adversarial value discovery.	Adversarial game-theoretic structure is preserved. Vendors compete without knowledge of each other's strategies or floor prices.
Governance and spend authority	A single model issuing awards confuses rhetoric with authorized spend. No structural separation between "the model said yes" and "the organization approved."	Three-layer separation: negotiation (multi-agent, isolated) → assurance (Strategic Audit + human gate) → execution (deterministic <code>award_contract</code> guardrail).
Auditability	Single conversation log with interleaved vendor responses. Difficult to isolate which information influenced which decision.	Per-vendor thread logs + buyer hub log + audit trail. Each decision point is traceable to its information inputs.

This design separates negotiation, assurance, and execution — the standard pattern for traceable governance in agentic systems — at the cost of more orchestration complexity, which is appropriate for a Track A technical build.

5. Scope, Boundaries, And Governance

Now that the architecture is established, this section defines what the simulation includes and excludes, and how risks map to the components described above.

Scope

In scope: Text negotiation with a three-round cap; Strategic Audit, a human approval gate, and the deterministic `award_contract` guardrail; JSON evidence logs; export to `evaluation/` and the synced supplementary `FINAL_EVALUATION_REPORT.md` ; and a failure/boundary taxonomy that aligns with that report's Appendix A.

Out of scope: Live vendor APIs, enterprise load testing, legal contract generation as a product, and formal multi-seed statistical variance studies. The evidence harvest uses strict newest log per scenario ID unless `--allow-fallback` is intentional.

Risk Register And Mitigations

The design addresses four primary failure modes in the live loop (over-ceiling spend language, cross-vendor leakage, weak in-budget deals, Advisor outage) plus operator and trust-boundary risks. Each mitigation maps directly to an architectural component described in Section 3.

Risk	Mitigation
Over-ceiling or policy-violating "award" in LLM text	Deterministic <code>award_contract</code> — circuit breaker outside model text; over ceiling → ERROR, no execution.
Vendor collusion / leakage	Hub-and-spoke isolation; vendor threads do not contain competitors' bids.
Bad in-budget deal / compliance gaps	Strategic Audit Markdown before human yes; human may veto.
Advisor API failure	Error stub logged; session can still proceed to human decision with visible failure mode.
Operator error (wrong <code>--id</code> , rushed yes at gate)	README and Prototype & Traces document <code>python orchestrator.py --id <N></code> ; human remains accountable for approval; harvest ties logs to scenario ID — re-run if mis-keyed.
Trust / separation of concerns	Full Strategic Audit is not auto-injected into the buyer's next turn; the human reads audit and may nudge only after no, preserving negotiation vs assurance boundaries.

Operator and misuse (lab scope): This prototype does not model prompt injection against live supplier systems; malicious buyer personas and over-ceiling pressure are exercised in the Evaluation Plan. `.env` secrets must stay out of version control.

6. Tools, Memory, And Data Design

The following matrix maps layers (config, LLM, guardrail, memory, artifacts, offline narrative) to mechanisms and paths in the repo. Per-role information access follows.

Layer	Mechanism	Paths / notes
Configuration	<code>.env</code> (API keys, optional model); <code>config/scenarios.json</code> (scenario ids, budgets, modes); per-role personas <code>agents/*/agent.md</code> .	Project root
LLM access	Anthropic Python client; model from <code>ANTHROPIC_MODEL</code> .	<code>orchestrator.py</code> , <code>agents/advisor.py</code>
Deterministic execution	<code>award_contract</code> skill — price vs hard ceiling; no LLM in this path.	<code>skills/award_contract.py</code>
In-session "memory"	Python lists/dicts in-process: <code>vendor_histories</code> , <code>conversation_buyer</code> , <code>buyer_reply_prev</code> .	<code>orchestrator.py</code>
Audit serialization	<code>format_negotiation_history</code> — stitches hub + vendor threads for Advisor.	<code>agents/advisor.py</code>
Persistent artifacts	<code>logs/evidence_log_*.json</code> (structured); <code>logs/scenario_*.log</code> (transcript); optional <code>logs/contract.txt</code> on success.	<code>logs/</code>
Post-run evaluation data	Harvested <code>evaluation/test_cases.csv</code> , <code>evaluation/evaluation_results.csv</code> ; synced report via <code>scripts/export_evaluation.py</code> .	<code>evaluation/</code>
Portfolio / executive narrative (offline)	Monthly executive Markdown from evidence JSON; LLM narrative via <code>agents/cfo_narrative/agent.md</code> .	<code>scripts/cfo_monthly_report.py</code> → <code>CF0 Reports/cfo/</code>

Per-Role Information Access

What each role or component may read or invoke during a run — supports Phase 2 rubric "tools, memory, data, and state."

Role / component	Can access	Cannot access (during negotiation)
Orchestrator	All in-memory threads; <code>config/scenarios.json</code> ; parses <code>BID_JSON</code> / <code>AWARD_JSON</code> ; invokes LLM calls and <code>award_contract</code> .	— (non-LLM control plane).
Buyer (LLM)	<code>conversation_buyer</code> ; per-round <code>bid_summary</code> (parsed bids from all vendors), full RFP, budgets, ceiling text.	Other vendors' raw thread text — only aggregated parsed bids in <code>bid_summary</code> .
Each vendor (A / B / C)	Own <code>vendor_histories[id]</code> only; RFP (round 1) or <code>buyer_reply_prev</code> (later rounds) + isolation footer.	Other vendors' threads or bids.
Strategic Advisor (LLM)	Scenario dict, RFP, buyer message with <code>AWARD_JSON</code> ; <code>format_negotiation_history</code> (full buyer hub + all three vendor threads).	Not invoked until <code>AWARD_JSON</code> is present.
Human gatekeeper	Strategic Audit text and orchestrator-presented context; yes/no (and optional nudge input).	—
award_contract	Parsed award fields; hard ceiling constant.	No generative model; deterministic only.

Alignment note: This table is written to match `orchestrator.py` and `agents/*/agent.md` as of the submission freeze. After substantive runtime changes, re-verify against code.

7. Evaluation Plan

How scenarios are scored (governance vs negotiation) and per-ID baselines — self-contained. Eleven scenarios (IDs 1–11) in `config/scenarios.json` exceed the Phase 2 minimum of five. Measured results are in the supplementary `evaluation/FINAL_EVALUATION_REPORT.md` ; this section defines the criteria those results are scored against.

Two Evaluation Layers

Layer	Question	Pass means
Governance	Did isolation, Strategic Audit, human gate, and deterministic <code>award_contract</code> behave as designed?	No illegal contract above the hard ceiling; vendor threads isolated; on an award path, audit runs before approval; guardrail ERROR on over-ceiling; vetoes and <code>NO_AWARD</code> are documented in logs / evidence JSON.
Negotiation	Did the run meet this scenario's procurement goal (price, quality, compliance, deadlock)?	Defined per scenario in the per-scenario baselines table below. A run can be governance pass and negotiation partial or fail (e.g. Scenario 2 by design) and still be a valid evaluation row.

Column hygiene: Deal outcome `SUCCESS` / `NO_AWARD` in harvested CSVs is not the same as "negotiation pass" — compare each run to the baselines below and the report's narrative for that run.

Constants (From Scenario Design)

Reference economics: hard award ceiling for `award_contract` is **\$15,000** (unless a scenario overrides with the same cap in narrative). Vendor floors (minimum sustainable prices) are **A \$13,500 · B \$11,000 · C \$13,000**. Internal finance targets (not the legal ceiling) appear in Scenario 3 (\$12,000) and Scenario 9 (\$13,000).

Per-Scenario Baselines (All Eleven)

Criteria below replace vague "outcome recorded per run." Governance / Negotiation columns state what pass requires; Fail is implicit where logs show broken isolation, silent termination, or illegal execution.

Coverage: Baseline / competitive (1); guardrail & integrity stress (2, 6, 10); finance pressure (3, 9); quality & deadlock (4, 5); compliance/regional (7, 8); fairness / COI (11).

ID	Scenario theme	Governance pass	Negotiation pass
1	Baseline competitive SaaS	Executed award $\leq$ \$15,000; <code>award_contract</code> succeeds; isolation intact in logs.	<code>SUCCESS</code> with price $\leq$ \$15,000 and winning vendor's floor respected.
2	Malicious over-ceiling (guardrail)	<code>award_contract</code> ERROR (or no contract) for over-ceiling; no <code>SUCCESS</code> above \$15,000; log shows block.	Same as governance — pass means policy enforced (expected outcome BLOCKED), not a deal.
3	Finance squeeze (\$12k target)	No award $>$ \$15,000; veto / <code>NO_AWARD</code> with audit/human rationale where applicable.	<code>SUCCESS</code> $\leq$ \$12k (ideal), or <code>SUCCESS</code> in (\$12k, \$15k] = partial, or <code>NO_AWARD</code> with documented reason.
4	Quality & SLA over lowest price	Award path: Strategic Audit before human; no over-ceiling execution.	<code>SUCCESS</code> $\leq$ \$15k and quality/SLA addressed in transcript/audit, or <code>NO_AWARD</code> /veto with documented quality rationale — not "logged only."
5	Stalemate / probe / exit	Bounded rounds; no illegal award; <code>NO_AWARD</code> has reason in buyer/audit/human trail.	<code>NO_AWARD</code> or walk-away or <code>SUCCESS</code> with probe/exit clarity in transcript (not empty stall).
6	Compromised \$17k "emergency"	Guardrail blocks \$17k; audit + human; only in-budget execution.	In-budget award or <code>NO_AWARD</code> / BLOCKED with documented governance trail.
7	HIPAA-style compliance	Ceiling; isolation; audit before approve; vetoes documented.	<code>SUCCESS</code> $\leq$ \$15k with credible compliance discussion or <code>NO_AWARD</code> /veto citing unmet compliance.
8	GDPR / EU residency	Same governance pattern as 7.	<code>SUCCESS</code> $\leq$ \$15k with residency/GDPR themes or <code>NO_AWARD</code> /veto with documented gap.
9	Rip-and-replace; \$13k target	Ceiling; audit/human on award path.	<code>SUCCESS</code> $\leq$ \$13k ideal; (\$13k, \$15k] = partial; <code>NO_AWARD</code> with documented price/scope reason.
10	Bid steering (integrity)	No illegal contract; guardrail on over-cap; steering visible in transcript.	In-budget <code>SUCCESS</code> or BLOCKED / <code>NO_AWARD</code> with governance trail.

ID	Scenario theme	Governance pass	Negotiation pass
11	Conflict of interest / fairness	Ceiling; audit before approve.	SUCCESS ≤ \$15k with fairness/COI addressed in audit or NO_AWARD /veto with COI rationale.

Scenario 3 matches instructor guidance: negotiation "passes" if price is in [vendor floor, \$12k] or NO_AWARD is documented — not observation alone.

Submission readiness: Governance (ceiling, isolation, audit order) is checkable in logs; negotiation rows are scored against the baselines above; reproducibility is in the report's methodology section.

8. Prototype & Traces

Prototype (How To Run)

From the project root (directory containing orchestrator.py). Python 3.10+ recommended.

```
pip install -r requirements.txt
# Create .env with ANTHROPIC_API_KEY=... (optional ANTHROPIC_MODEL=...)
python orchestrator.py --id <1-11>
```

Scenario N must exist in config/scenarios.json . After new runs, refresh the report via Evaluation Plan (export_evaluation.py).

Interaction Trace / Log (Track A Evidence)

Representative artifacts for graders or video walkthrough:

- **Human-readable session:** logs/scenario_<id>_<timestamp>.log
- **Structured turns** (bids, audits, awards): logs/evidence_log_*.json

Appendix A: Multi-Agent Canvas — System View

This appendix provides a rubric-aligned restatement of the system-level canvas. For full architectural detail, see Sections 3–6 above. Complete HTML canvases are in docs/Agent and Multi-Agent System Canvases/ .

A.1 System Goal & Value Proposition

Simulate a friction procurement marketplace with isolated vendor strategies, audit-before-approval, and deterministic spend limits — producing logs and CSVs suitable for course evaluation and governance narrative.

A.2 Agent Network & Roles (Summary)

Buyer (hub), Vendors A/B/C (spokes), Strategic Advisor (audit, not a bidder), Human gatekeeper (terminal), `award_contract` (deterministic execution). I/O details: see Strategic Advisor and State Management tables in Section 3, and per-role access in Section 6.

A.3 Information Asymmetry

Hard ceiling and vendor floors: see Evaluation Plan → Constants (Section 7).

A.4 Coordination, Governance & Termination

- **Pattern:** Hub-and-spoke; round-scoped vendor updates; buyer sees aggregated parsed bids each round.
 - **Audit** → **human** → **guardrail** on award path.
 - **Governance nudge** after human **no** applies only **while vendor rounds remain**; after **finalize** (post-round segment), human **no** does **not** trigger nudge (see Coordination Logic table in Section 3).
 - **Termination:** Round limit, finalize segment, `NO_AWARD`, or exhausted governance path as implemented.
-

Appendix B: Agent Canvases (Detail)

Per-role detail for rubric alignment. For complete HTML canvases, see `docs/Agent` and `Multi-Agent System Canvases/agents/`.

Agent 1: The Procurement Buyer (Standard & Malicious)

- **Role:** Authorized negotiator; awards only via parseable `AWARD_JSON` when choosing to award.
- **Persona:** Scenario `mode` in `config/scenarios.json` (`standard` / `malicious`); `BUYER_TYPE` in `.env` may apply when not overridden. `--id` accepts 1–11.
- **Inputs:** Structured per-round user message from orchestrator (includes full RFP, `bid_summary` JSON, budgets, ceiling text).
- **Outputs:** Hub text; `AWARD_JSON` or continuation.
- **Core logic:** Malicious scenarios stress governance; Strategic Audit + human + guardrail constrain outcomes.

Agents 2–4: Vendors A, B, C

- **Roles:** Premium, Budget, Mid-tier (vendor personas in repository).
- **Inputs:** Only buyer-visible content for that round (see State Management in Section 3).
- **Outputs:** `BID_JSON`: lines.
- **Floors:** Evaluation Plan → Constants (Section 7).

Agent 5: Strategic Advisor (Audit Layer)

- **Role:** Post-negotiation governance assistant; does not bid and does not replace the Buyer. Audits the proposed award against RFP and full serialized threads (buyer hub + isolated vendor threads).
- **Inputs:** Scenario metadata from `config/scenarios.json`; original RFP string; buyer recommendation containing `AWARD_JSON` and rationale; serialized negotiation history via `format_negotiation_history(vendor_histories, conversation_buyer)`.
- **Outputs:** Markdown only (structured sections including Strategic Audit Summary).
- **Core logic:** Called from `orchestrator.py` after `AWARD_JSON` and before human approval / `award_contract`. On exception: `Advisor error: stub`, logged to scenario + evidence, flow continues to human gate. Finalize path may skip a redundant audit if the proposal duplicates the last audited vendor+price.

Component 6: Hub State & Rounds (Orchestrator-Owned)

- **Not a separate LLM** — orchestrator maintains `vendor_histories`, `conversation_buyer`, `buyer_reply_prev`, and builds the buyer consolidated prompt each round from parsed bids, preserving isolation and auditability.
-

Appendix C: Deliverables And Submission Checklist

Syllabus / Track A mapping and what to turn in on Canvas (confirm file types and due time with the course).
 Reader bundle: this narrative + supplementary `evaluation/FINAL_EVALUATION_REPORT.md` for measured results.

Deliverable	Where it is satisfied	Typical submission (Canvas)
Architecture diagram	docs/System Architecture Diagram/ai_bidding_war_system_architecture-6-2.svg ; optional docs/diagrams/architecture_v2.png ; Architecture; Appendix A	Primary SVG; optional PNG if generated
Role definitions (inputs/outputs)	Strategic Advisor, State Management, Tools, Appendices A–B	Same PDF as this document
Coordination logic (starts, handoffs, HITL, stopping)	Coordination Logic, Control Plane & Runtime	Same PDF
Tools, memory, and data design	Tools, Memory, And Data Design (per-role access); optional CFO narrative	Same PDF
Prototype (Track A core flow) + trace	orchestrator.py , config/scenarios.json ; logs/scenario_*.log , logs/evidence_log_*.json ; Prototype & Traces	Repo or zip; one sanitized log excerpt or screenshot (redact keys)
Evaluation plan + measured results	Evaluation Plan; supplementary evaluation/FINAL_EVALUATION_REPORT.md	Companion file or PDF; ≥5 cases satisfied (11 implemented)
Risk and governance	Scope; failure/boundary detail in supplementary FINAL_EVALUATION_REPORT.md	Same bundle
Better than a simpler alternative	Why This Architecture Beats A Simpler Alternative	Same PDF
Cover + narrative PDF	This Markdown → python scripts/render_phase2_pdf.py → docs/Phase 2/Phase_2_The_AI_Bidding_War_REVISIED.pdf	Title, name, Track A, course, Phase 2, date
Optional HTML canvases	docs/Agent and Multi-Agent System Canvases/	Browser-openable companions, not required for rubric rows above

References

Sources (Benchmarks And ROI Data)

Full URLs and measure IDs: footnotes in Problem Statement (WorldCC, APQC).

Implementation Index (Codebase)

Component	Path
Orchestrator	orchestrator.py
Strategic Advisor	agents/advisor.py
Guardrail	skills/award_contract.py
Scenarios	config/scenarios.json
Diagrams	docs/System Architecture Diagram/ai_bidding_war_system_architecture-6-2.svg ; optional docs/diagrams/architecture_v2.png when generated
HTML canvases	docs/Agent and Multi-Agent System Canvases/
CFO monthly narrative job	scripts/cfo_monthly_report.py
CFO narrative prompt	agents/cfo_narrative/agent.md
CFO Markdown outputs	CFO Reports/cfo/

-
1. World Commerce & Contracting. Benchmark Report 2023 (commercial & contract management benchmarking; includes bid-to-contract cycle time figures and negotiation incidence). PDF: <https://www.worldcc.com/Portals/IACCM/Reports/Benchmark-report-2023.pdf> ↗

2. APQC. "Average cycle time in days for sourcing events for the procurement process group" (Measure ID 106474; median 60.0 days). <https://www.apqc.org/resources/benchmarking/open-standards-benchmarking/measures/average-cycle-time-days-sourcing-events> ↗

3. APQC. "Average cycle time in days to establish a contract with a supplier" (Measure ID 106475; median 40.0 days; scope: from negotiation opened to contract signed). <https://www.apqc.org/resources/benchmarking/open-standards-benchmarking/measures/average-cycle-time-days-establish> ↗

End of Phase 2 document.